Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai) Bhavan's Campus, Munshi Nagar,
Andheri (West), Mumbai-400058-India

ADVANCED DATABASE PROGRAMMING

Quick Commerce Database Business Rules and Operations

Stored procedures, triggers and views

Student	Batch	UID
Aryan Kadu	B2	2025300103
Trisha Khandelwal	B2	2025300112
Siddhesh Khanolkar	B2	2025300114

Submission date: 20/10/26 | Database: MySQL 8.0+ | Project: QuickCommerceDB | Phase III

Repository: https://github.com/gitdorah/quick-commerce-database

Introduction

This assignment adds stored procedures, triggers and views to the existing MySQL schema so that order reservations and cancellations update inventory consistently, invalid quantities are rejected, and operational information can be queried through reusable views. The implementation is designed around the central business rule that available stock must never be oversold, including when multiple checkouts compete for the same inventory row.

Objectives

- Implement transactional procedures to place and cancel customer orders.
- Use triggers to reject nonpositive order quantities and negative inventory balances.
- Create views for available stock, order summaries and low-stock monitoring.
- Demonstrate expected behavior with positive, boundary and failure-case SQL tests.
- Explain transaction boundaries, concurrency behavior and the limits of the current teaching schema.

Database scope and assumptions

The work uses Customer, DarkStore, Product, InventoryState, Inventory, Order and OrderItem from the earlier assignments. Order is quoted with backticks because it is a MySQL reserved word. Each Inventory row is a balance for one dark store, product and state; the existing unique key on (DarkStoreID, ProductID, StateID) allows safe upsert behavior. Stock is moved between Available and Reserved when a single-product order is placed, and returned to Available when a confirmed order is cancelled. Payment, delivery, reservation expiry and inventory movement history remain outside this implementation.

1. Business rules and implementation map

Database object	Rule or responsibility
PlaceOrder	Requires a positive quantity and sufficient Available stock. Moves units to Reserved and creates a Confirmed order and its order line in one transaction.
CancelOrder	Accepts only an existing Confirmed order. Releases each order line from Reserved, restores Available stock and marks the order Cancelled atomically.

ValidateOrderItem	Rejects inserted order lines whose Quantity is NULL or less than or equal to zero.
ValidateInventoryInsert / ValidateInventoryUpdate	Rejects NULL or negative Inventory.Quantity values on insert or update.
AvailableInventoryView	Reports each product-store pair and its Available quantity; missing Available rows display as zero.
OrderSummaryView	Combines order, customer, dark store and item totals for reporting.
LowStockView	Lists product-store pairs whose AvailableQuantity is below the example threshold of 10 units.

The procedures do more than wrap SQL statements: they establish the transaction boundary, define how failures are signaled, and keep the stock movement and order status consistent. The triggers protect basic data invariants even when a caller writes directly to the underlying tables.

2. Stored procedures

2.1 PlaceOrder

PlaceOrder accepts CustomerID, DarkStoreID, ProductID and Quantity. It first rejects invalid quantity input, then starts a transaction. A conditional UPDATE reserves stock only when the matching Available row has at least the requested quantity. InnoDB serializes competing updates to that row; after waiting for another transaction, the predicate is evaluated against the current value. A zero affected-row count causes a SIGNAL and the exception handler rolls back all changes. Only after reservation succeeds does the procedure increase the Reserved balance, insert the order and insert its order item.

```
-- ------------------------------------------------------------------------
-- PROCEDURE 1: PlaceOrder
-- PURPOSE:
-- Creates an order and reserves inventory for one product.
--
-- BUSINESS RULES:
-- 1. Quantity must be positive.
-- 2. Available inventory cannot fall below zero.
-- 3. Available stock decreases and reserved stock increases atomically.
-- 4. If any operation fails, the transaction is rolled back.
-- ------------------------------------------------------------------------
DROP PROCEDURE IF EXISTS PlaceOrder$$

CREATE PROCEDURE PlaceOrder(
    IN p_CustomerID INT,
    IN p_DarkStoreID INT,
    IN p_ProductID INT,
    IN p_Quantity INT
)
BEGIN
    DECLARE v_AvailableStateID INT;
    DECLARE v_ReservedStateID INT;
    DECLARE v_OrderID INT;

    -- Roll back every change if any SQL error occurs.
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    -- Validate the requested quantity.
    IF p_Quantity IS NULL OR p_Quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Order quantity must be positive';
    END IF;

    START TRANSACTION;

    -- Retrieve the inventory-state identifiers.
```

```
    SELECT StateID INTO v_AvailableStateID
    FROM InventoryState
    WHERE StateName = 'Available';

    SELECT StateID INTO v_ReservedStateID
    FROM InventoryState
    WHERE StateName = 'Reserved';

    -- Atomically reserve stock only if sufficient stock exists.
    -- Concurrent requests must compete for the same inventory row.
    UPDATE Inventory
    SET Quantity = Quantity - p_Quantity
    WHERE DarkStoreID = p_DarkStoreID
      AND ProductID = p_ProductID
      AND StateID = v_AvailableStateID
      AND Quantity >= p_Quantity;

    -- An affected-row count of zero means the reservation failed.
    IF ROW_COUNT() = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Insufficient available stock or inventory record missing';
    END IF;

    -- Increase reserved stock, creating the row if necessary.
    INSERT INTO Inventory
        (DarkStoreID, ProductID, StateID, Quantity)
    VALUES
        (p_DarkStoreID, p_ProductID, v_ReservedStateID, p_Quantity)
    ON DUPLICATE KEY UPDATE
        Quantity = Quantity + p_Quantity;

    -- Create the order only after the inventory reservation succeeds.
    INSERT INTO `Order`
        (OrderDate, Status, CustomerID, DarkStoreID)
    VALUES
        (NOW(), 'Confirmed', p_CustomerID, p_DarkStoreID);

    SET v_OrderID = LAST_INSERT_ID();

    -- Create the order line.
    INSERT INTO OrderItem (OrderID, ProductID, Quantity)
    VALUES (v_OrderID, p_ProductID, p_Quantity);

    COMMIT;

    -- Return the newly created order ID.
    SELECT
        v_OrderID AS OrderID,
        'Confirmed' AS OrderStatus,
        p_Quantity AS QuantityReserved;
END$$
```

This check-and-decrement is one atomic database operation. The following Reserved upsert and order writes share the same transaction, so a later SQL error does not leave stock reserved without an order. The procedure returns the new OrderID, Confirmed status and quantity reserved.

2.2 CancelOrder

CancelOrder locks the order row with SELECT ... FOR UPDATE, checks that the order exists and is Confirmed, then iterates through its order items. Each item is removed from Reserved only when enough reserved units exist; an upsert adds the same quantity to Available. Once all lines are processed, the status changes to Cancelled and the transaction commits. A second cancellation is rejected because the status is no longer Confirmed. The order-row lock serializes attempts to cancel the same order.

```
-- ------------------------------------------------------------------------
```

```
-- PROCEDURE 2: CancelOrder
-- PURPOSE:
-- Cancels a confirmed order and returns its reserved stock to availability.
--
-- BUSINESS RULES:
-- 1. Only confirmed orders can be cancelled.
-- 2. Reserved stock is reduced by the order-item quantities.
-- 3. Available stock is restored.
-- 4. All changes are committed together or rolled back together.
-- ------------------------------------------------------------------------
DROP PROCEDURE IF EXISTS CancelOrder$$

CREATE PROCEDURE CancelOrder(
    IN p_OrderID INT
)
BEGIN
    DECLARE v_OrderStatus VARCHAR(30);
    DECLARE v_DarkStoreID INT;
    DECLARE v_ProductID INT;
    DECLARE v_Quantity INT;
    DECLARE v_ReservedStateID INT;
    DECLARE v_AvailableStateID INT;
    DECLARE v_Done BOOLEAN DEFAULT FALSE;

    -- Process every product belonging to the order.
    DECLARE order_items CURSOR FOR
        SELECT ProductID, Quantity
        FROM OrderItem
        WHERE OrderID = p_OrderID;

    -- End cursor processing when no more order items remain.
    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET v_Done = TRUE;

    -- Roll back all modifications if an SQL error occurs.
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Lock the order row and retrieve its status and store.
    SET v_Done = FALSE;

    SELECT Status, DarkStoreID
    INTO v_OrderStatus, v_DarkStoreID
    FROM `Order`
    WHERE OrderID = p_OrderID
    FOR UPDATE;

    IF v_Done = TRUE THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Order does not exist';
    END IF;

    IF v_OrderStatus <> 'Confirmed' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Only confirmed orders can be cancelled';
    END IF;

    SELECT StateID INTO v_ReservedStateID
    FROM InventoryState
    WHERE StateName = 'Reserved';
```

```
    SELECT StateID INTO v_AvailableStateID
    FROM InventoryState
    WHERE StateName = 'Available';

    OPEN order_items;

    item_loop: LOOP
        FETCH order_items INTO v_ProductID, v_Quantity;

        IF v_Done THEN
            LEAVE item_loop;
        END IF;

        -- Remove the cancelled quantity from reserved stock.
        UPDATE Inventory
        SET Quantity = Quantity - v_Quantity
        WHERE DarkStoreID = v_DarkStoreID
          AND ProductID = v_ProductID
          AND StateID = v_ReservedStateID
          AND Quantity >= v_Quantity;

        IF ROW_COUNT() = 0 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT =
                    'Reserved inventory is insufficient for cancellation';
        END IF;

        -- Restore the cancelled quantity to available stock.
        INSERT INTO Inventory
            (DarkStoreID, ProductID, StateID, Quantity)
        VALUES
            (v_DarkStoreID, v_ProductID,
             v_AvailableStateID, v_Quantity)
        ON DUPLICATE KEY UPDATE
            Quantity = Quantity + v_Quantity;
    END LOOP;

    CLOSE order_items;

    -- Mark the order as cancelled.
    UPDATE `Order`
    SET Status = 'Cancelled'
    WHERE OrderID = p_OrderID;

    COMMIT;

    SELECT
        p_OrderID AS OrderID,
        'Cancelled' AS OrderStatus;
END$$

DELIMITER ;
```

The cursor supports multi-item orders even though the current PlaceOrder procedure creates one order line per call. Releasing stock and changing status in one transaction prevents partially cancelled orders.

3. Triggers and views

3.1 Data validation triggers

ValidateOrderItem is a BEFORE INSERT trigger that raises SQLSTATE 45000 when a new order line has NULL or nonpositive quantity. ValidateInventoryInsert and ValidateInventoryUpdate apply the same rejection pattern to inventory balances

below zero. This makes direct table edits obey core invariants as well as changes made by the stored procedures. The earlier table constraints remain useful as a second layer of validation.

```
  -- ------------------------------------------------------------------------
-- TRIGGER 1: ValidateOrderItem
-- PURPOSE:
-- Rejects invalid order-item quantities before insertion.
-- ------------------------------------------------------------------------

DROP TRIGGER IF EXISTS ValidateOrderItem$$

CREATE TRIGGER ValidateOrderItem
BEFORE INSERT ON OrderItem
FOR EACH ROW
BEGIN
    IF NEW.Quantity IS NULL OR NEW.Quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Order item quantity must be greater than zero';
    END IF;
END$$


-- ------------------------------------------------------------------------
-- TRIGGER 2: ValidateInventoryInsert
-- PURPOSE:
-- Prevents negative inventory quantities during insertion.
-- ------------------------------------------------------------------------

DROP TRIGGER IF EXISTS ValidateInventoryInsert$$

CREATE TRIGGER ValidateInventoryInsert
BEFORE INSERT ON Inventory
FOR EACH ROW
BEGIN
    IF NEW.Quantity IS NULL OR NEW.Quantity < 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Inventory quantity cannot be negative';
    END IF;
END$$


-- ------------------------------------------------------------------------
-- TRIGGER 3: ValidateInventoryUpdate
-- PURPOSE:
-- Prevents inventory updates from producing negative quantities.
-- ------------------------------------------------------------------------

DROP TRIGGER IF EXISTS ValidateInventoryUpdate$$

CREATE TRIGGER ValidateInventoryUpdate
BEFORE UPDATE ON Inventory
FOR EACH ROW
BEGIN
    IF NEW.Quantity IS NULL OR NEW.Quantity < 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT =
                'Inventory quantity cannot be negative';
    END IF;
END$$

DELIMITER ;
```

3.2 Reporting views

AvailableInventoryView CROSS JOINs the store and product catalog, then LEFT JOINs the Available balance. This intentionally includes product-store combinations with no Available row and maps them to zero with COALESCE. OrderSummaryView aggregates order lines and computes units and value using the current Product.Price. LowStockView reuses AvailableInventoryView and filters quantities below 10.

OrderSummaryView is an operational summary, not a historical invoice: because OrderItem does not store UnitPriceAtOrder, a later catalogue price change can change the displayed total for an old order. A production extension should snapshot the charged unit price on each order line. The low-stock threshold is a configurable example rule rather than a universal replenishment policy.

```
-- ----------------------------------------------------------------------------
-- VIEW 1: AvailableInventoryView
-- PURPOSE:
-- Shows available stock for every product at every dark store.
-- Products with no Available inventory row appear with quantity zero.
-- ----------------------------------------------------------------------------

CREATE OR REPLACE VIEW AvailableInventoryView AS
SELECT
    ds.DarkStoreID,
    ds.Location AS DarkStoreLocation,
    p.ProductID,
    p.Name AS ProductName,
    p.Price,
    COALESCE(i.Quantity, 0) AS AvailableQuantity
FROM DarkStore AS ds
CROSS JOIN Product AS p
LEFT JOIN InventoryState AS s
    ON s.StateName = 'Available'
LEFT JOIN Inventory AS i
    ON i.DarkStoreID = ds.DarkStoreID
   AND i.ProductID = p.ProductID
   AND i.StateID = s.StateID;


-- ----------------------------------------------------------------------------
-- VIEW 2: OrderSummaryView
-- PURPOSE:
-- Combines order, customer, dark-store, and order-item information.
-- Calculates the order total using the current catalogue prices.
-- ----------------------------------------------------------------------------

CREATE OR REPLACE VIEW OrderSummaryView AS
SELECT
    o.OrderID,
    o.OrderDate,
    o.Status,
    c.CustomerID,
    c.Name AS CustomerName,
    ds.DarkStoreID,
    ds.Location AS DarkStoreLocation,
    COUNT(oi.OrderItemID) AS NumberOfOrderItems,
    COALESCE(SUM(oi.Quantity), 0) AS TotalUnits,
    COALESCE(SUM(oi.Quantity * p.Price), 0.00) AS OrderTotal
FROM `Order` AS o
INNER JOIN Customer AS c
    ON o.CustomerID = c.CustomerID
INNER JOIN DarkStore AS ds
    ON o.DarkStoreID = ds.DarkStoreID
LEFT JOIN OrderItem AS oi
    ON o.OrderID = oi.OrderID
LEFT JOIN Product AS p
    ON oi.ProductID = p.ProductID
GROUP BY
```

```
    o.OrderID,
    o.OrderDate,
    o.Status,
    c.CustomerID,
    c.Name,
    ds.DarkStoreID,
    ds.Location;


-- ------------------------------------------------------------------------
-- VIEW 3: LowStockView
-- PURPOSE:
-- Identifies product-store combinations with fewer than 10 available units.
-- The threshold is an example business rule and can be adjusted.
-- ------------------------------------------------------------------------

CREATE OR REPLACE VIEW LowStockView AS
SELECT
    DarkStoreID,
    DarkStoreLocation,
    ProductID,
    ProductName,
    AvailableQuantity
FROM AvailableInventoryView
WHERE AvailableQuantity < 10;
```

4. Demonstration and expected results

Run the database creation, table creation and sample-data scripts first, then execute advanced_database_programming.sql in QuickCommerceDB. The following statements are interactive checks; run the cancellation using the OrderID returned by PlaceOrder rather than assuming a fixed identifier.

```
-- Test 1: View available inventory.
SELECT *
FROM AvailableInventoryView
ORDER BY DarkStoreID, ProductID;
```

DarkStoreID	DarkStoreLocation	ProductID	ProductName	Price	AvailableQuantity
1	Andheri	1	Amul Taaza Milk 1L	62.00	25
1	Andheri	2	Britannia Bread 400g	45.00	40
1	Andheri	3	Lay's Classic Salted 50g	20.00	60
1	Andheri	4	Coca Cola 750ml	40.00	35
1	Andheri	5	Tata Salt 1kg	30.00	0
1	Andheri	6	Maggi 2-Minute Noodles 70g	15.00	50
1	Andheri	7	Surf Excel Matic 2kg	320.00	0
1	Andheri	8	Parle-G Biscuits 800g	85.00	0
2	Bandra	1	Amul Taaza Milk 1L	62.00	20
2	Bandra	2	Britannia Bread 400g	45.00	30
2	Bandra	3	Lay's Classic Salted 50g	20.00	45
2	Bandra	4	Coca Cola 750ml	40.00	25
2	Bandra	5	Tata Salt 1kg	30.00	35
2	Bandra	6	Maggi 2-Minute Noodles 70g	15.00	0
2	Bandra	7	Surf Excel Matic 2kg	320.00	0
2	Bandra	8	Parle-G Biscuits 800g	85.00	0
3	Powai	1	Amul Taaza Milk 1L	62.00	18
3	Powai	2	Britannia Bread 400g	45.00	25
3	Powai	3	Lay's Classic Salted 50g	20.00	0
3	Powai	4	Coca Cola 750ml	40.00	0
3	Powai	5	Tata Salt 1kg	30.00	40
3	Powai	6	Maggi 2-Minute Noodles 70g	15.00	0
3	Powai	7	Surf Excel Matic 2kg	320.00	15
3	Powai	8	Parle-G Biscuits 800g	85.00	0
4	Dadar	1	Amul Taaza Milk 1L	62.00	0
4	Dadar	2	Britannia Bread 400g	45.00	0
4	Dadar	3	Lay's Classic Salted 50g	20.00	50
4	Dadar	4	Coca Cola 750ml	40.00	0
4	Dadar	5	Tata Salt 1kg	30.00	30
4	Dadar	6	Maggi 2-Minute Noodles 70g	15.00	45
4	Dadar	7	Surf Excel Matic 2kg	320.00	0
4	Dadar	8	Parle-G Biscuits 800g	85.00	25

32 rows in set (0.001 sec)

```
-- Test 2: View order summaries.
SELECT *
FROM OrderSummaryView
ORDER BY OrderID;
```

OrderID	OrderDate	Status	CustomerID	CustomerName	DarkStoreID	DarkStoreLocation	NumberOfOrderItems	TotalUnits	OrderTotal
1	2026-09-30 09:15:00	Delivered	1	Aarav Mehta	1	Andheri	2	3	169.00
2	2026-09-30 09:22:00	Preparing	2	Riya Sharma	2	Bandra	2	5	140.00
3	2026-09-30 09:31:00	Confirmed	3	Kabir Shah	1	Andheri	2	5	122.00
4	2026-09-30 09:42:00	Out for Delivery	4	Ananya Patel	3	Powai	2	3	380.00
5	2026-09-30 09:55:00	Confirmed	5	Vihaan Joshi	4	Dadar	2	3	190.00

5 rows in set (0.002 sec)

```
-- Test 3: Identify low-stock products.
SELECT *
FROM LowStockView
ORDER BY AvailableQuantity, DarkStoreLocation;
```

```
+-------------+-------------------+-----------+----------------------------+-------------------+
| DarkStoreID | DarkStoreLocation | ProductID | ProductName                | AvailableQuantity |
+-------------+-------------------+-----------+----------------------------+-------------------+
|           1 | Andheri           |         5 | Tata Salt 1kg              |                 0 |
|           1 | Andheri           |         7 | Surf Excel Matic 2kg       |                 0 |
|           1 | Andheri           |         8 | Parle-G Biscuits 800g      |                 0 |
|           2 | Bandra            |         6 | Maggi 2-Minute Noodles 70g |                 0 |
|           2 | Bandra            |         7 | Surf Excel Matic 2kg       |                 0 |
|           2 | Bandra            |         8 | Parle-G Biscuits 800g      |                 0 |
|           4 | Dadar             |         1 | Amul Taaza Milk 1L         |                 0 |
|           4 | Dadar             |         2 | Britannia Bread 400g       |                 0 |
|           4 | Dadar             |         4 | Coca Cola 750ml            |                 0 |
|           4 | Dadar             |         7 | Surf Excel Matic 2kg       |                 0 |
|           3 | Powai             |         3 | Lay's Classic Salted 50g   |                 0 |
|           3 | Powai             |         4 | Coca Cola 750ml            |                 0 |
|           3 | Powai             |         6 | Maggi 2-Minute Noodles 70g |                 0 |
|           3 | Powai             |         8 | Parle-G Biscuits 800g      |                 0 |
+-------------+-------------------+-----------+----------------------------+-------------------+
14 rows in set (0.001 sec)
```

```
-- Test 4: Place an order for two units of milk at Andheri.
-- CustomerID = 1, DarkStoreID = 1, ProductID = 1.
-- Place an order for two units of milk
CALL PlaceOrder(1, 1, 1, 2);
```

```
mysql> -- CustomerID = 1, DarkStoreID = 1, ProductID = 1.
Query OK, 0 rows affected (0.000 sec)

mysql> -- Expected: a new confirmed order and two units reserved.
Query OK, 0 rows affected (0.000 sec)

mysql> CALL PlaceOrder(1, 1, 1, 2);
+---------+-------------+-------------------+
| OrderID | OrderStatus | QuantityReserved |
+---------+-------------+-------------------+
|       6 | Confirmed   |                 2 |
+---------+-------------+-------------------+
1 row in set (0.004 sec)

Query OK, 0 rows affected (0.004 sec)
```

```
-- Test 5: Inspect the latest order.
SELECT *
FROM OrderSummaryView
ORDER BY OrderID DESC
LIMIT 1;
```

```
+---------+---------------------+-----------+------------+--------------+-------------+-------------------+----------------------+------------+-------------+
| OrderID | OrderDate           | Status    | CustomerID | CustomerName | DarkStoreID | DarkStoreLocation | NumberOfOrderItems | TotalUnits | OrderTotal |
+---------+---------------------+-----------+------------+--------------+-------------+-------------------+----------------------+------------+-------------+
|       6 | 2026-10-20 00:49:09 | Confirmed |          1 | Aarav Mehta  |           1 | Andheri           |                    1 |          2 |      124.00 |
+---------+---------------------+-----------+------------+--------------+-------------+-------------------+----------------------+------------+-------------+
1 row in set (0.001 sec)

mysql>
```

```
SELECT *
FROM AvailableInventoryView
WHERE DarkStoreID = 1
  AND ProductID = 1;
```

```
+-------------+-------------------+-----------+--------------------+-------+-------------------+
| DarkStoreID | DarkStoreLocation | ProductID | ProductName        | Price | AvailableQuantity |
+-------------+-------------------+-----------+--------------------+-------+-------------------+
|           1 | Andheri           |         1 | Amul Taaza Milk 1L | 62.00 |                23 |
+-------------+-------------------+-----------+--------------------+-------+-------------------+
1 row in set (0.002 sec)
```

With the supplied sample data, milk at Andheri begins with 25 Available units. A successful two-unit order should reduce that balance to 23 and increase Reserved by two. The latest order summary should show a new Confirmed order, one order item, two units and a current-price total of 124.00. LowStockView should report only balances below 10, including zero-balance product-store pairs because the availability view includes missing rows.

```
-- Use the OrderID returned by PlaceOrder
CALL CancelOrder(wrongID);
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'To test cancellation, use the OrderID returned by Pl
aceOrder:

-- Available milk should return to its pre-order balance
SELECT * FROM AvailableInventoryView
WHERE DarkStoreID = 1 AND ProductID = 1;
+-------------+-------------------+-----------+--------------------+-------+-------------------+
| DarkStoreID | DarkStoreLocation | ProductID | ProductName        | Price | AvailableQuantity |
+-------------+-------------------+-----------+--------------------+-------+-------------------+
|           1 | Andheri           |         1 | Amul Taaza Milk 1L | 62.00 |                23 |
+-------------+-------------------+-----------+--------------------+-------+-------------------+
1 row in set (0.002 sec)

-- The trigger should reject this update
UPDATE Inventory SET Quantity = -5 WHERE InventoryID = 1;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'Then test the validation trigger:
```

After cancellation, the order should show Cancelled, Available milk should return to 25 and Reserved should return to its earlier balance. The negative update should fail with the trigger message and leave inventory unchanged. Replace the placeholder with the numeric OrderID returned by the procedure.

5. Validation plan

Test	Action	Expected behavior
Successful reservation	Place an order with a positive quantity no greater than Available stock.	Order and order line are created; Available decreases and Reserved increases by the requested amount.
Insufficient stock	Request more units than are currently Available.	Procedure raises an error; transaction rollback leaves balances and orders unchanged.
Invalid order quantity	Call PlaceOrder with 0 or a negative quantity; separately try inserting an invalid OrderItem.	Procedure validation or trigger rejects the request; no order is created.
Cancellation	Cancel a Confirmed order and query the order and inventory views.	Order becomes Cancelled; stock moves back from Reserved to Available.
Repeat cancellation	Call CancelOrder again for the same OrderID.	Request is rejected because only Confirmed orders may be cancelled.
Negative inventory update	Set an Inventory.Quantity to a negative value.	BEFORE UPDATE trigger rejects the change; the original balance remains intact.
Concurrent checkout	Run competing PlaceOrder calls for the same product/store when stock is limited.	Conditional updates serialize against the same row; successful reservations cannot exceed stock.

The SQL statements define reproducible checks; the expected results above are derived from the supplied sample data and procedure logic. Record actual MySQL output and error messages when the script is run in the database environment. A read-only SELECT from AvailableInventoryView is useful for inspection but cannot reserve stock or guarantee that stock remains available after the query.

6. Limitations and possible extensions

- PlaceOrder creates one product line per order invocation. A multi-product checkout procedure could accept a staging table or JSON item list, validate all lines, lock inventory rows in a stable order and commit the order as one unit.
- The implementation assumes InnoDB transactional tables and relies on MySQL row locking and affected-row behavior. Deadlock victims should be retried by the application using a bounded retry policy.
- No payment confirmation or reservation timeout is modeled. A production workflow would define when Reserved stock is consumed, released on payment failure or expired, and how repeated requests are made idempotent.
- Inventory balances show current quantity but do not provide an audit trail. An append-only inventory-movement table would capture order, cancellation, adjustment and replenishment events.
- The views calculate order value from current catalogue prices. Store UnitPriceAtOrder and currency on order lines for stable historical reporting.

Conclusion

The advanced database programming layer adds reusable operations and database-enforced safeguards to the normalized quick-commerce schema. PlaceOrder and CancelOrder keep order state and inventory balances within atomic transactions; conditional updates protect the available-stock boundary under concurrent checkouts. Triggers reject invalid quantities, while views give consistent read models for stock, order summaries and low-stock review. These mechanisms turn the design's integrity requirements into executable database behavior and provide a foundation for extending the teaching system toward multi-item checkout, payment handling and inventory auditing.